

Tutorial de Angular: Do Zero à Criação de um Projeto

1. Instalação do Node.js e npm

Node.js é um ambiente de execução de JavaScript no lado do servidor, que permite executar código JavaScript fora de um navegador. **npm** (Node Package Manager) é o gerenciador de pacotes do Node.js, usado para instalar e gerenciar pacotes de software.

Passos para Instalação:

- **Windows/Mac/Linux:**
 1. Acesse o site oficial [Node.js](https://nodejs.org/).
 2. Baixe a versão LTS (Long Term Support) recomendada.
 3. Siga as instruções do instalador para concluir a instalação.

Verificar a Instalação:

No terminal, execute os comandos:

- `node -v`
Exibe a versão do Node.js instalada.
- `npm -v`
Exibe a versão do npm instalada.

Comandos Úteis do npm:

1. `npm install <package>`
Instala um pacote localmente no projeto.
2. `npm install -g <package>`
Instala um pacote globalmente no sistema.
3. `npm update <package>`
Atualiza um pacote instalado.
4. `npm uninstall <package>`
Remove um pacote instalado.
5. `npm init`
Inicializa um novo projeto Node.js, criando um arquivo `package.json`.

2. Instalação do Angular CLI

Angular CLI é uma ferramenta de linha de comando para inicializar, desenvolver e manter aplicativos Angular.

Instalação:

Para instalar o Angular CLI globalmente, execute o comando:

```
npm install -g @angular/cli
```

se o terminal acusar erro e não instalar
execute os comandos

```
npm config set strict-ssl false
```

```
npm config set registry http://registry.npmjs.org/
```

E como admin do power shell execute
Set-ExecutionPolicy Unrestricted

Verificar a Instalação:

No terminal, execute:

- `ng version`
Exibe a versão do Angular CLI instalada.

Principais Comandos do Angular CLI:

Criar um Novo Projeto:

```
ng new nome-do-projeto
```

1. Esse comando cria um novo projeto Angular com a estrutura de diretórios e arquivos essenciais.

Rodar o Servidor de Desenvolvimento:

```
ng serve
```

2. Inicia um servidor de desenvolvimento local, permitindo que você visualize o aplicativo no navegador (geralmente em <http://localhost:4200>).

Criar um Componente:

```
ng generate component nome-do-componente
```

3. Gera um novo componente Angular com arquivos `.ts`, `.html`, `.css`, e `.spec.ts`.

Criar um Serviço:

```
ng generate service nome-do-servico
```

4. Cria um serviço Angular, que é usado para lógica de negócios e comunicação com APIs.

Build para Produção:

```
ng build --prod
```

5. Compila o projeto para produção, otimizando os arquivos para serem servidos em um ambiente de produção.

3. Exemplo Prático: Criando um Projeto Angular**Criar um novo projeto:**

```
ng new meu-primeiro-projeto
```

Escolha as opções desejadas durante a criação, como adicionar o roteamento e o estilo CSS preferido.

Navegar até o diretório do projeto:

```
cd meu-primeiro-projeto
```

Rodar o servidor de desenvolvimento:

```
ng serve
```

Abra <http://localhost:4200> no navegador para ver o projeto rodando.

Criar um componente de exemplo:

```
ng generate component exemplo
```

Criar um serviço de exemplo:

```
ng generate service exemplo
```

Projeto criado em sala consulta-cep

Criar projeto consulta-cep

```
ng new consulta-cep
```

Criar componente cep

```
ng generate component cep
```

ou

```
ng g c cep
```

criar serviço cep

```
ng generate service cep/cep
```

ou

```
ng g s cep/cep
```

Obs: Relembrando que pode ser abreviado alguns comandos:

generate -> abreviado fica: g

componente -> abreviado fica: c

Logo:

```
ng g c cep
```

Consulta no serviço viacep

Criação do serviço cep.

Criação de um atributo http do tipo HttpClient.

No construtor é usado o conceito de injeção de dependência para a biblioteca HttpClient.

Método consultaCep faz a requisição e retorna uma promise.

```
1  import { HttpBackend, HttpClient } from '@angular/common/http';
2  import { Injectable } from '@angular/core';
3
4  @Injectable({
5    providedIn: 'root'
6  })
7  export class CepService {
8
9    private http: HttpClient;
10
11    constructor(
12      handler: HttpBackend
13    ) {
14      this.http = new HttpClient(handler);
15    }
16
17    public consultaCep(){
18      //cep https://viacep.com.br/ws/85807594/json
19      let url = 'https://viacep.com.br/ws/';
20
21      return this.http.get(url + 85807594 + '/json').toPromise();
22    }
23  }
24
25 }
```



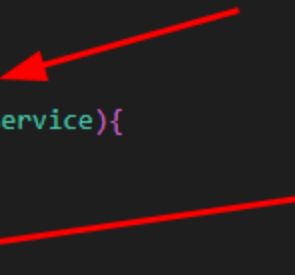
Componente consumindo o serviço consultaCep.

No construtor realizar a injeção de dependência do CepService

```

4  @Component({
5      selector: 'app-cep',
6      imports: [],
7      templateUrl: './cep.component.html',
8      styleUrls: ['./cep.component.css']
9  })
10 export class CepComponent {
11
12     constructor(private cepService: CepService){
13
14     }
15
16     public buscarCep(){
17         this.cepService.consultaCep().then( resposta => {
18             let retorno: any = resposta;
19             alert(retorno.logradouro);
20         });
21     }
22
23 }
24

```



Componente HTML executando o serviço consultaCep

```

<button (click)="buscarCEP()">Buscar CEP</button>

```

App.config lembrar de “cadastrar” o serviço como provider (provedor) ou seja, deixá-lo disponível para toda a aplicação.

```

7 | import { HttpClient, provideHttpClient } from '@angular/common/http';
8
9 | export const appConfig: ApplicationConfig = {
10 |   providers:
11 |     [provideZoneChangeDetection({ eventCoalescing: true }),
12 |      provideRouter(routes),
13 |      provideClientHydration(withEventReplay()),
14 |      provideHttpClient()]
15 |   ];
16 | };
17

```

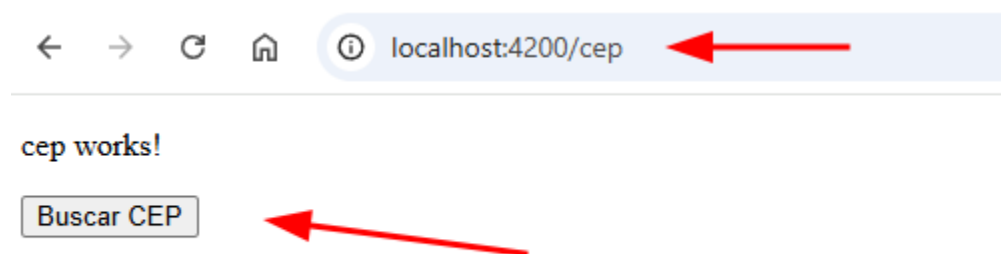
Criar uma rota para o CEP

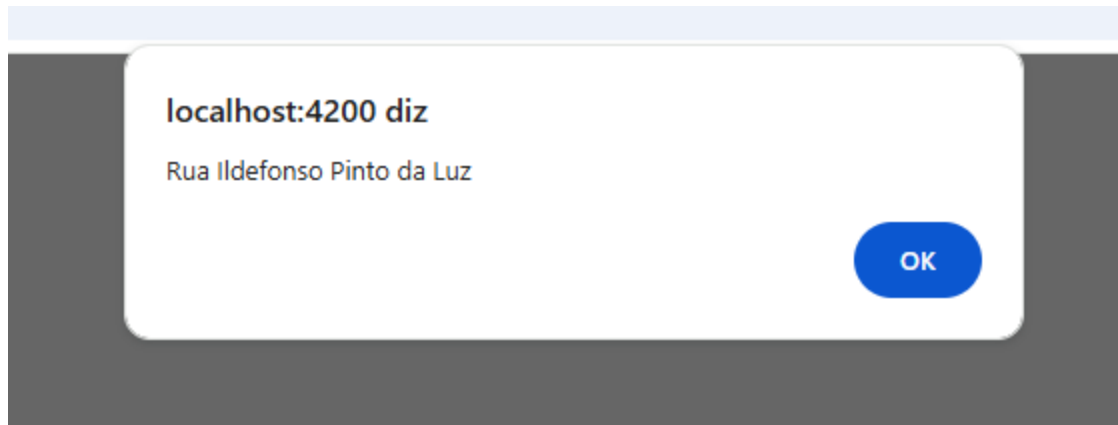
```

src > app > TS app.routes.ts > routes > path
1 | import { Routes } from '@angular/router';
2 | import { CepComponent } from '../cep/cep.component';
3
4 | export const routes: Routes = [
5 |   {path: 'cep', component: CepComponent}
6 | ];
7

```

No navegador testar a rota





Atividade

Fazer funcionar o serviço de CEP configurando o formulário para digitar o CEP e buscar automaticamente os demais dados.

Estilizar as páginas HTML usando CSS

Criar outro componente para posts

<https://jsonplaceholder.typicode.com/posts>

Buscar da API acima os dados e exibir em tela

Usar um laço de repetição, para criar dinamicamente as tags HTML.

Ex:

Se na API tiver 100 posts

para cada post uma estrutura pode ser criada como abaixo:

```
<div class="post">
  <p>{{title}}</p>
  <p>{{body}}</p>
</div>
```

Dúvidas procurar o professor.